

Mediastream Cheat Sheet

SQL + PostgreSQL + pipeline analytics. Condensado para entrevista tecnica: CRUD, joins, agregaciones, time-series, indices, JSONB, Kafka y respuestas cortas.

DDL + CRUD

```
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  activo BOOLEAN DEFAULT true,  
  salario NUMERIC(10,2),  
  creado_en TIMESTAMP DEFAULT NOW()  
);  
  
ALTER TABLE usuarios ADD COLUMN edad INT;  
ALTER TABLE usuarios ALTER COLUMN nombre TYPE  
VARCHAR(200);  
ALTER TABLE usuarios DROP COLUMN activo;
```

```
INSERT INTO usuarios (nombre, email)  
VALUES ('Erick', 'erick@email.com')  
RETURNING id;  
  
SELECT nombre, salario  
FROM usuarios  
WHERE salario > 3000  
ORDER BY salario DESC  
LIMIT 10 OFFSET 0;  
  
UPDATE usuarios SET salario = salario * 1.1 WHERE id  
= 1;  
DELETE FROM usuarios WHERE id = 1;
```

GROUP BY + HAVING

```
SELECT departamento,  
  COUNT(*) AS total,  
  ROUND(AVG(salario), 2) AS sueldo_medio  
FROM empleados  
GROUP BY departamento  
HAVING AVG(salario) > 2500;
```

Top content ultimos 7 dias

```
SELECT content_id,  
  COUNT(*) AS plays,  
  SUM(duration_sec) AS total_duration_sec  
FROM events  
WHERE event_type = 'play'  
  AND created_at >= NOW() - INTERVAL '7 days'  
GROUP BY content_id  
ORDER BY plays DESC, content_id ASC  
LIMIT 5;
```

FILTROS UTILES

```
WHERE nombre LIKE 'Er%'  
WHERE nombre ILIKE 'er%'  
WHERE creado_en BETWEEN '2026-01-01' AND '2026-12-31'  
WHERE id IN (1,2,3,4)
```

Time-series / analytics

```
SELECT DATE_TRUNC('hour', created_at) AS hour,  
  COUNT(*) AS plays  
FROM events  
WHERE created_at >= NOW() - INTERVAL '24 hours'  
GROUP BY 1  
ORDER BY 1;
```

Para dashboards: casi siempre filtrar por rango de fechas, agrupar por hora/dia y devolver { rows, total, meta }.

JOINS

```
SELECT u.nombre, p.total  
FROM usuarios u  
INNER JOIN pedidos p ON u.id = p.usuario_id;
```

```
SELECT u.nombre, p.total  
FROM usuarios u  
LEFT JOIN pedidos p ON u.id = p.usuario_id;
```

Regla mental

- **INNER**: solo coincidencias.
- **LEFT**: mantener tabla izquierda aunque falten hijos.
- En analytics: evitar N+1; preferir query agregada + indice correcto.

POSTGRES QUE SUMA PUNTOS

```
INSERT INTO usuarios (id, nombre, email)
VALUES (1, 'Erick Ramos', 'erick@email.com')
ON CONFLICT (id)
DO UPDATE SET nombre = EXCLUDED.nombre;
```

```
CREATE TABLE proyectos (
  id SERIAL PRIMARY KEY,
  datos JSONB
);

SELECT datos->>'nombre' FROM proyectos;
SELECT * FROM proyectos
WHERE datos->'tags' @> '["react"]';
```

```
SELECT DISTINCT ON (departamento)
  departamento, nombre, salario
FROM empleados
ORDER BY departamento, salario DESC;
```

RENDIMIENTO

```
CREATE INDEX idx_usuarios_email ON usuarios(email);

CREATE INDEX idx_events_created_content
ON events(created_at, content_id);
```

```
EXPLAIN ANALYZE
SELECT *
FROM usuarios
WHERE email = 'erick@email.com';
```

- Indexar columnas de WHERE, JOIN y ordenacion frecuente.
- Para metrics: comunes (created_at, content_id), (country_code, created_at).
- Si el dashboard recalcula siempre lo mismo: materialized views / pre-aggregates.

WINDOW FUNCTIONS

```
SELECT *
FROM (
  SELECT content_id,
         country_code,
         COUNT(*) AS plays,
         ROW_NUMBER() OVER (
           PARTITION BY country_code
           ORDER BY COUNT(*) DESC
         ) AS rn
  FROM events
  GROUP BY content_id, country_code
) t
WHERE rn <= 3;
```

ROW_NUMBER **RANK** **PARTITION BY** **DATE_TRUNC**

MONGO VS POSTGRES

- **Postgres**: joins, agregaciones SQL, time-series, consistencia.
- **MongoDB**: documentos flexibles, metadata anidada, esquemas cambiantes.
- **Tu frase**: "SQL es mi día a día; Mongo lo usaría para eventos o read models flexibles."

Kafka / pipeline

```
[Player SDK] -> [Ingest API] -> [Kafka]
-> [Consumer/Aggregator] -> [Postgres/Mongo]
-> [API] -> [React dashboard]
```

- Kafka desacopla ingestion de lectura y suaviza picos.
- Consumer idempotente + at-least-once.
- Si no operaste Kafka: mencionar colas/workers/webhooks como analogía real.

RESPUESTAS CORTAS DE ENTREVISTA

- 1) Por que cola y no escribir directo a DB?
Porque desacopla ingestion del consumo, absorbe picos, permite retries, reprocesamiento e independientes readers sin bloquear el API principal.
- 2) Donde pre-agregarías?
Por hora / content / country. El dashboard lee tablas agregadas y no eventos crudos.
- 3) Que mostrar si el pipeline va 2 min atrasado?
Ultimo timestamp procesado + badge "data delayed" para transparencia.
- 4) Tu historia fuerte?
Pluvia: 100k+ filas, ag-Grid SSRM, contrato API estable, filtros tipicos <300 ms.
- 5) Tu gap honesto?
No opere Kafka en prod, pero entiendo el patron y ya trabaje con APIs, workers, validacion, observabilidad y read models en Postgres.